

Unit No. 2

Process Management

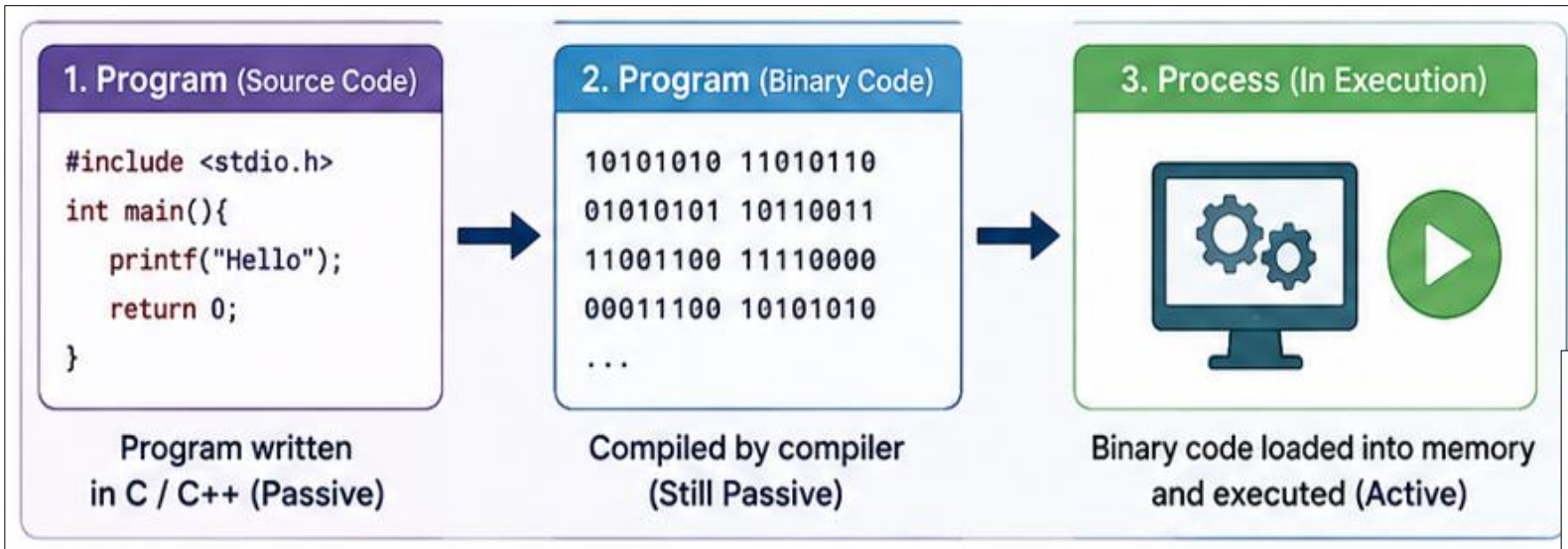
Course Outcome 1

- Understand processes, deadlock, context switch, and threads.

Content:

- Process Management: Process concept, Process States, process control block, context switch,
- Process scheduling: Scheduling Queues, Schedulers.
- IPC: Inter-process communication, Examples of IPC systems.
- Thread: concept, Multithreading models, Types of threads.

Process Concept

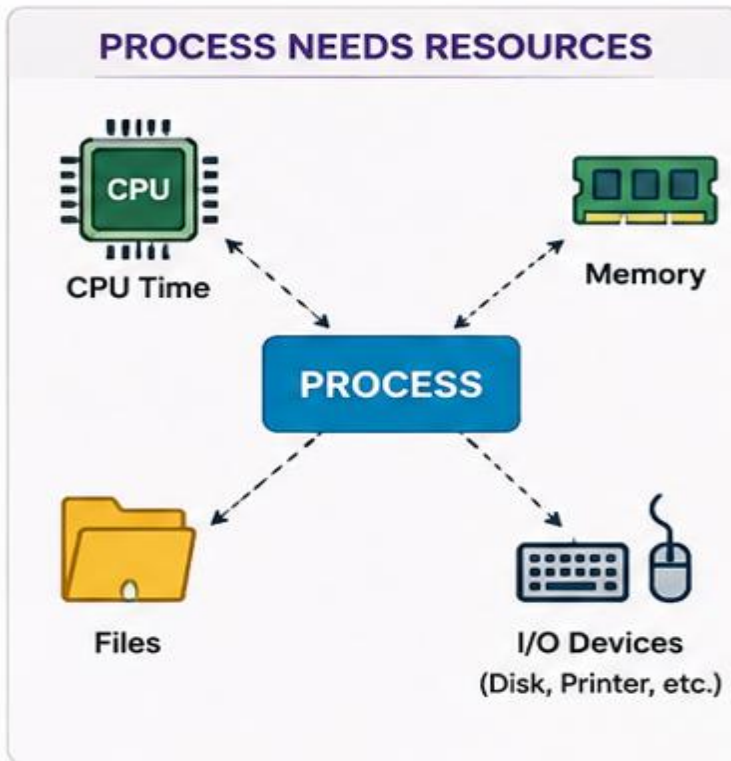


- A **program in execution** is called a process.
 - For example, when we write a program in C or C++ and compile it, the compiler creates binary code. The original code and binary code are both programs. When we actually run the binary code, it becomes a process.

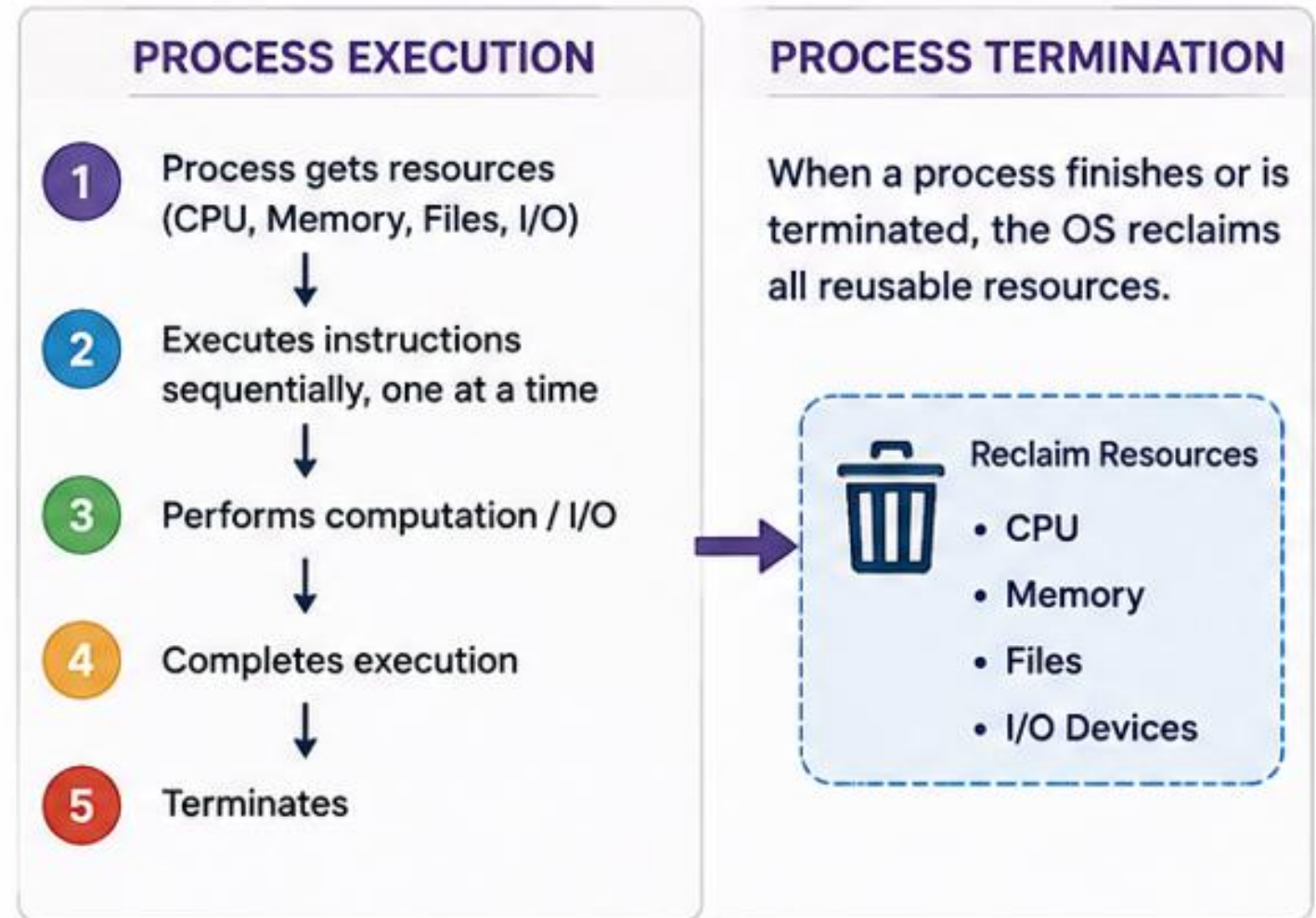
KEY POINTS

- A program in execution is called a process.
- Program = Passive entity (stored on disk)
- Process = Active entity (in memory & running)
- A program becomes a process when an executable file is loaded into memory.
- It is a unit of work within the system.

Process Concept



A process needs resources, including CPU time, Memory, files & I/O devices, to accomplish its task.



Process Concept

Task Manager

File Options View

Processes Performance App history Startup Users Details Services

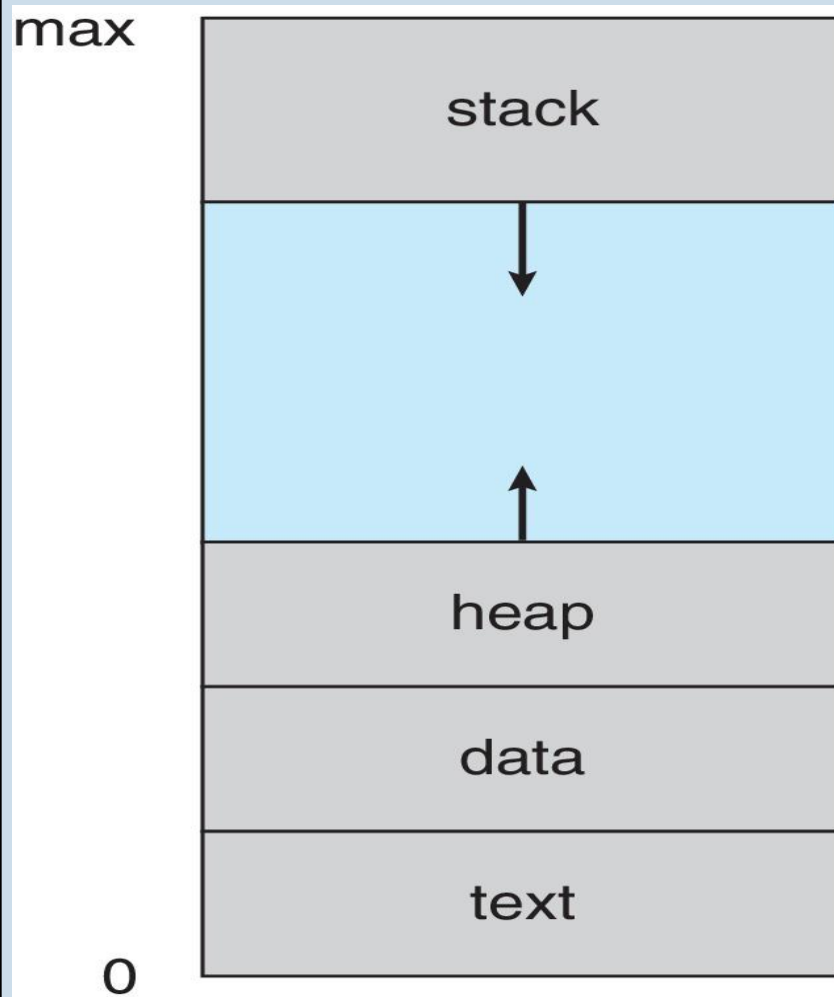
Name	Status	3% CPU	67% Memory	3% Disk	0% Network	P
Apps (4)						
> Google Chrome (32 bit) (18)		0%	590.8 MB	0.1 MB/s	0.1 Mbps	
> Microsoft PowerPoint (32 bit) (2)		0.3%	55.5 MB	0 MB/s	0 Mbps	
> Task Manager		2.3%	24.1 MB	0 MB/s	0 Mbps	
> Windows Explorer		0.9%	45.8 MB	0 MB/s	0 Mbps	
Background processes (39)						
> Acrobat Update Service (32 bit)		0%	0.3 MB	0 MB/s	0 Mbps	
> Antimalware Service Executable		0%	148.1 MB	0 MB/s	0 Mbps	
COM Surrogate		0%	1.3 MB	0 MB/s	0 Mbps	
CTF Loader		0%	2.1 MB	0 MB/s	0 Mbps	
Google Crash Handler		0%	0.1 MB	0 MB/s	0 Mbps	
Google Crash Handler (32 bit)		0%	0.4 MB	0 MB/s	0 Mbps	
Host Process for Windows Tasks		0%	1.4 MB	0 MB/s	0 Mbps	

< >

^ Fewer details End task

- A single program can create many processes when run multiple times.
- For example, when we open a **.exe** or **binary file** multiple times, multiple instances begin (multiple processes are created).

Process Concept:-



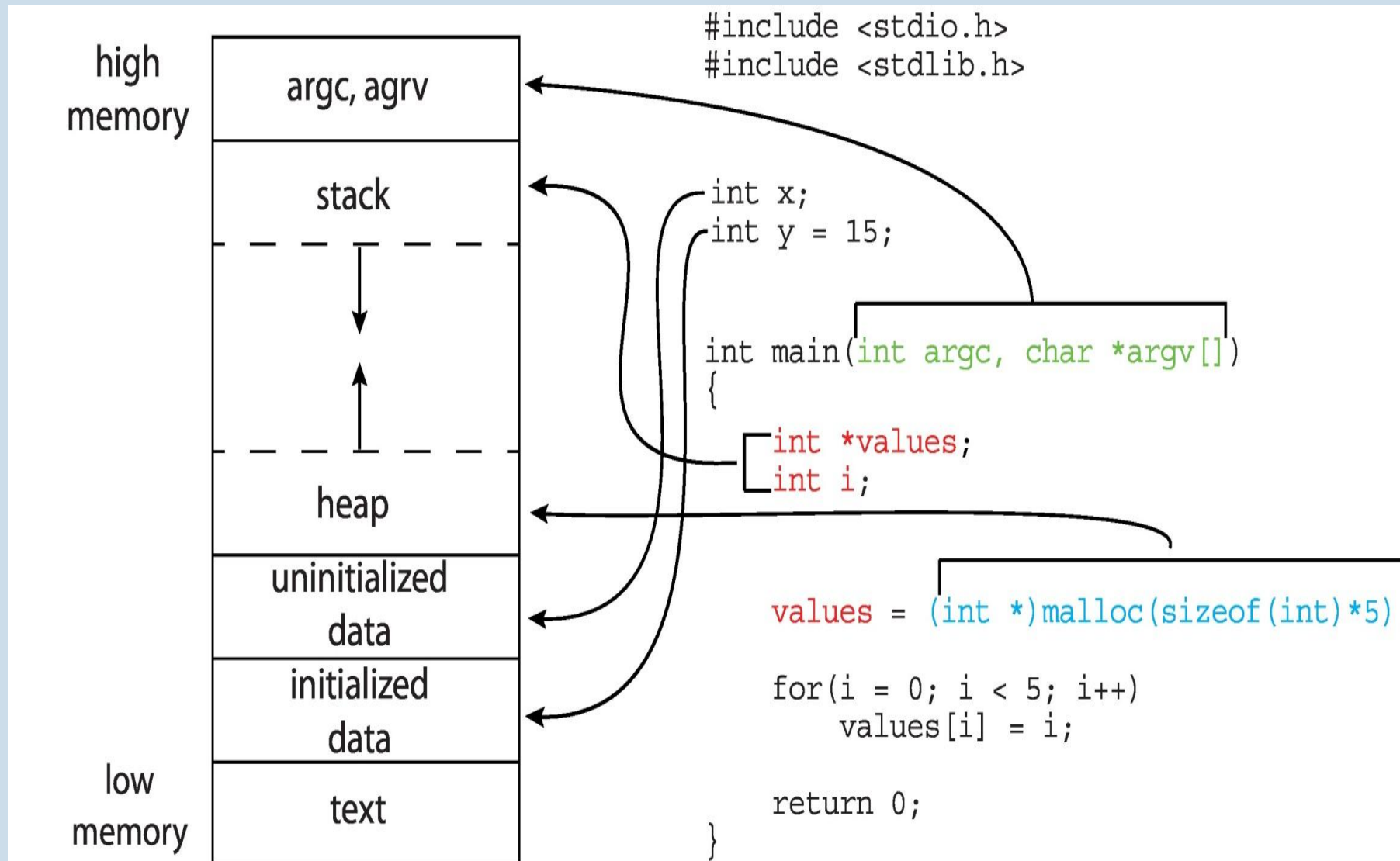
Multiple parts

- The program code, also called **text section**
- Current activity including **program counter**, processor registers
- **Stack** containing temporary data
 - Function parameters, return addresses, local variables
- **Data section** containing global variables
- **Heap** containing memory dynamically allocated during run time

Process in Memory

Process Concept:-

Memory Layout of a C Program

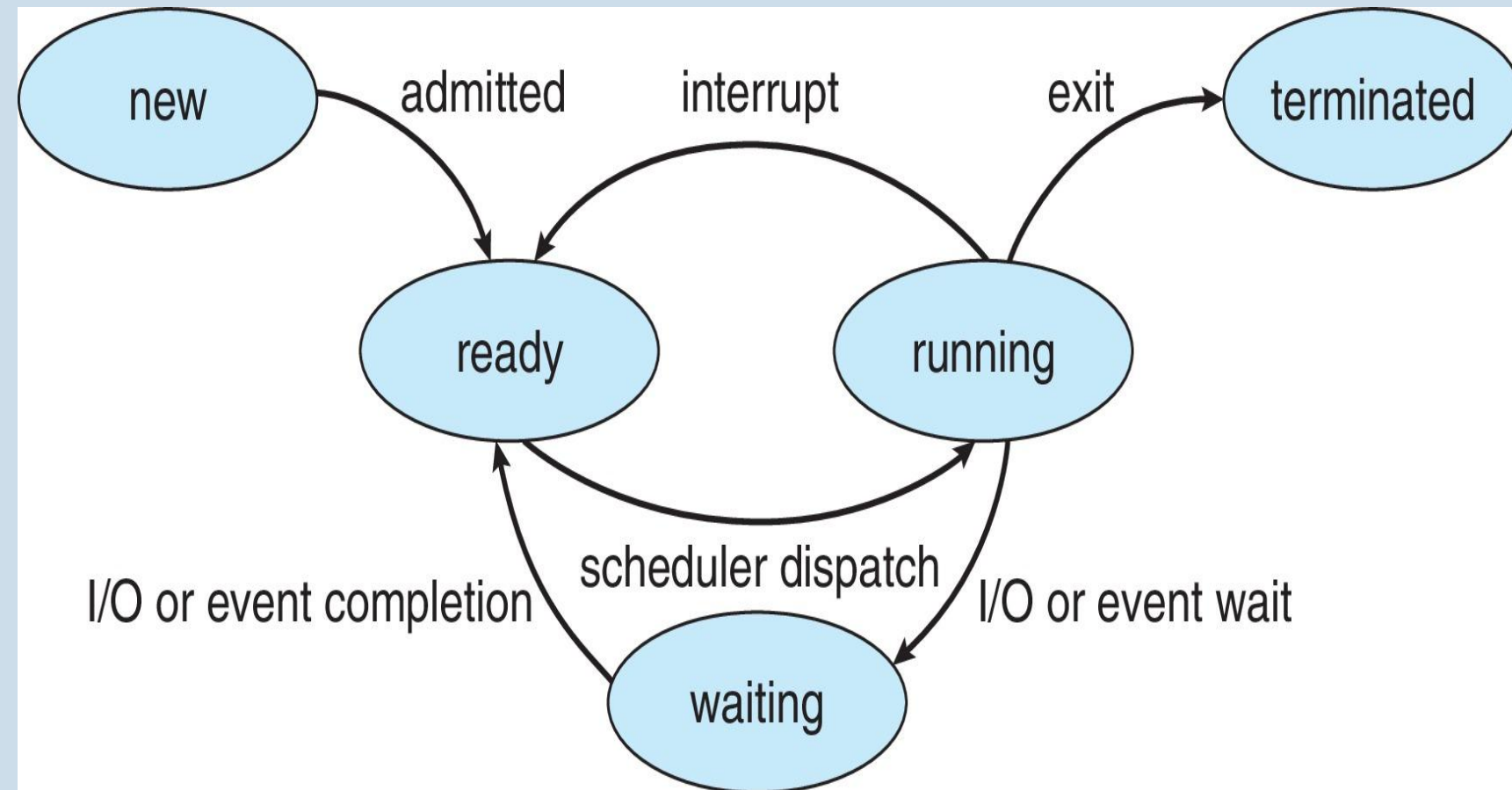


Process management :-

- The OS is responsible for the following activities in connection with processes managed.
 - The creation and deletion of both user and system processes
 - The suspension and resumption of processes.
 - The provision of mechanisms for process synchronization
 - Process communication
 - The provision of mechanisms for deadlock handling

Process State :-

- The process, from its creation to completion, passes through various states.
- The minimum number of states is five.
 1. New
 2. Ready
 3. Running
 4. Block or wait
 5. Completion or termination



Process State :-

1. New

- A program which is going to be picked up by the OS into the main memory is called a new process.

2. Ready

- Whenever a process is created, it directly enters in the ready state, in which, it waits for the CPU to be assigned.
- The OS picks the new processes from the secondary memory and put all of them in the main memory.
- The processes which are ready for the execution and reside in the main memory are called ready state processes.

3. Running

- One of the processes from the ready state will be chosen by the OS depending upon the scheduling algorithm.

Process State :-

4. Wait (or Block):

- When a process requests I/O access.
- When a process waits for a certain resource to be assigned or for the input from the user then the OS move this process to the block or wait state and assigns the CPU to the other processes.

5. Complete (or Terminated):

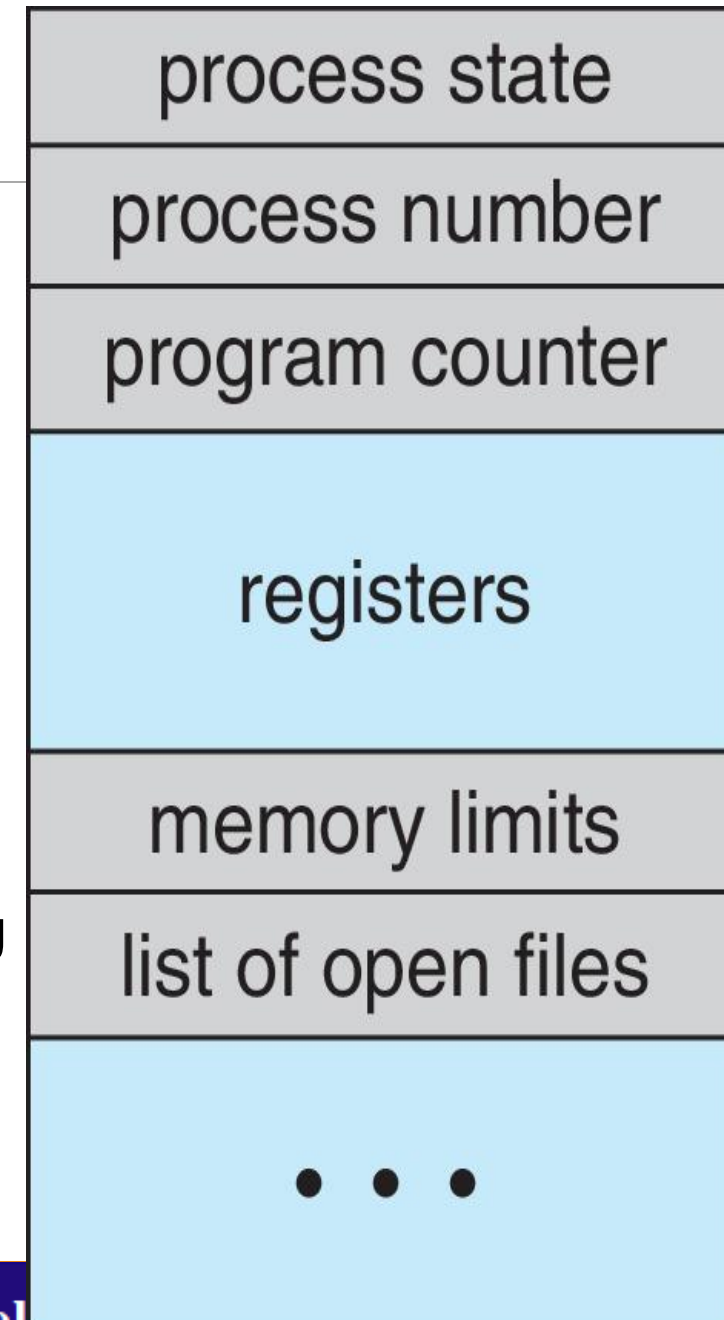
- When a process finishes its execution, it comes in the termination state.
- All the context of the process (Process Control Block) will also be deleted the process will be terminated by the Operating system.
- **Suspended Ready:** When the ready queue becomes full, some processes are moved to a suspended ready state
- **Suspended Block:** When the waiting queue becomes full.

Process Control Block (PCB)

- **Process Control Block is a data structure that contains information of the process related to it.**

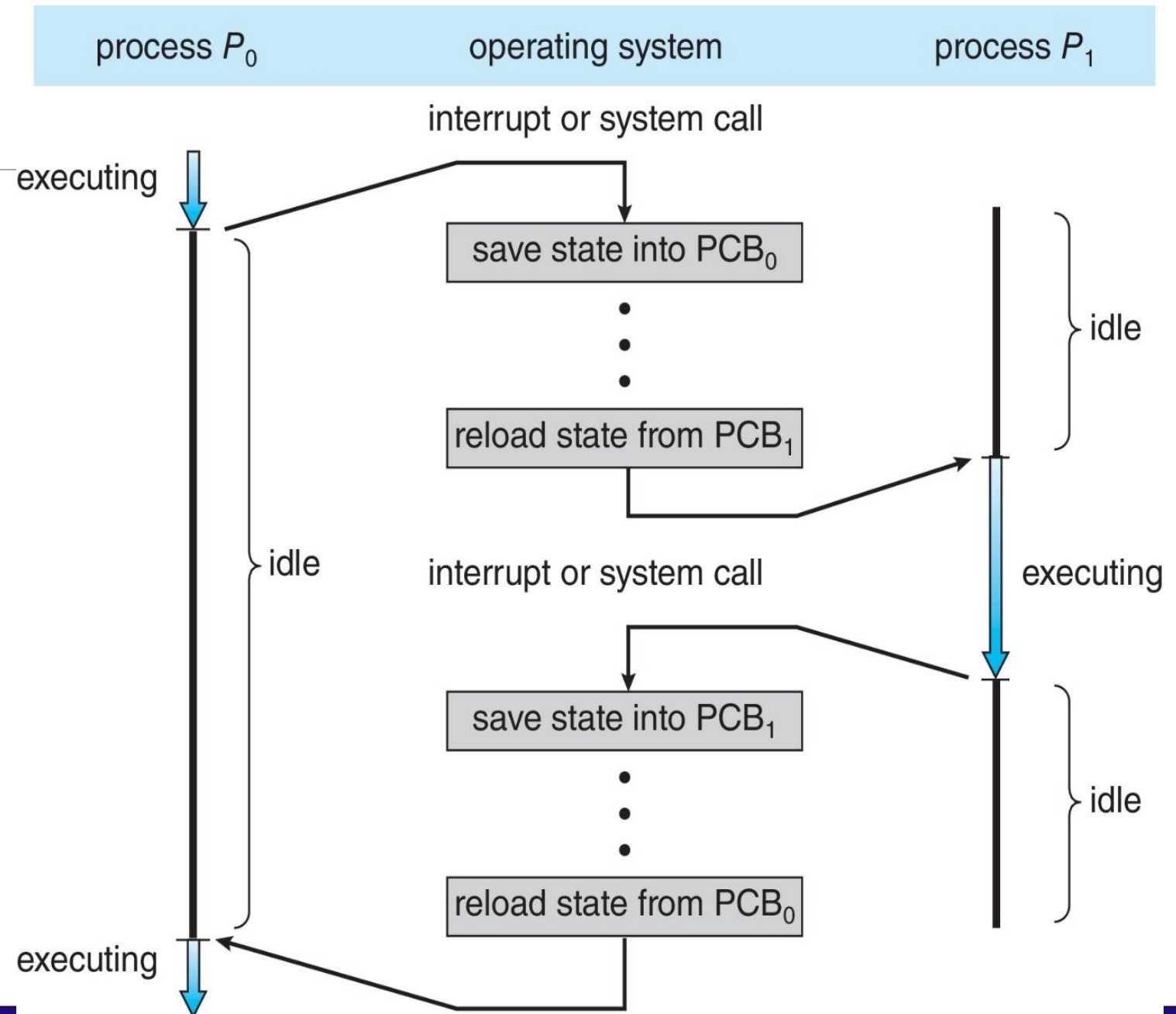
Information associated with each process(also called task control block)

- **Process state** – The state may be new, ready, running, and waiting, halted, and so on.
- **Program counter** – The counter indicates the address of the next instruction to be executed for this process.
- **CPU registers** – contents of all process-centric registers-They include accumulators, index registers, stack pointers, and general-purpose registers, plus any condition-code information.
- **CPU scheduling information-** priorities, scheduling queue pointers
- **Memory-management information** – memory allocated to the process
- **Accounting information** – This information includes the amount of CPU and real time used, time limits, account numbers, job or process numbers, and so on.
- **I/O status information** – I/O devices allocated to process, list of open files



Context Switch

A **context switch** occurs when the CPU switches from one process to another.



Context Switch

When CPU switches to another process, the system must **save the state** of the old process and load the **saved state** for the new process via a **context switch**

Context of a process represented in the PCB

Context-switch time is pure overhead; the system does no useful work while switching

- The more complex the OS and the PCB → the longer the context switch

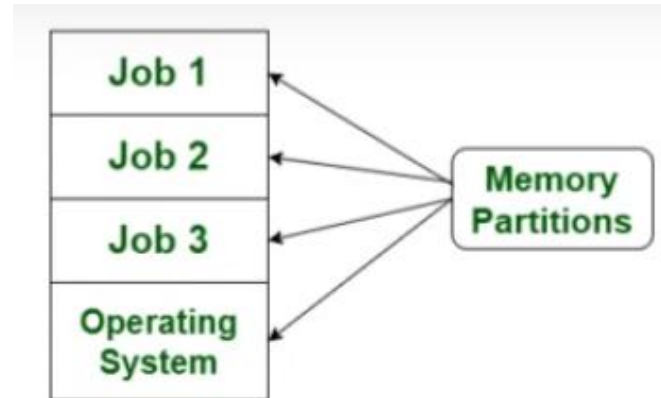
Time dependent on hardware support

- Some hardware provides multiple sets of registers per CPU → multiple contexts loaded at once

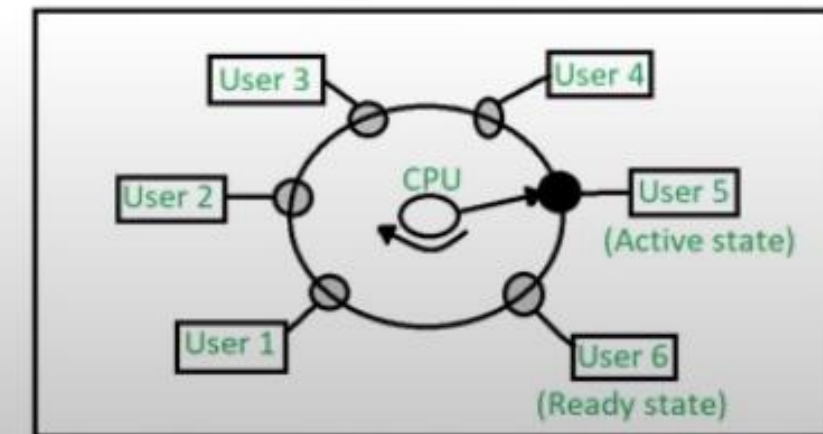
Process Scheduling

- The objective of multiprogramming is to have some process running at all times, to maximize CPU utilization.
- The objective of time sharing is to switch the CPU among processes so frequently that users can interact with each program while it is running.
- To meet these objectives, the **process scheduler** selects an available process to execute on the CPU.
- For a single-processor system, there will never be more than one running process.
- If there are more processes, the rest will have to wait until the CPU is free and can be rescheduled.
- Eg. Running Java Programming, Media Player & Chrome to download eclipse at the same time

Multiprogramming OS



Time Sharing OS



Process Scheduling

Process scheduler selects among available processes for next execution on CPU core

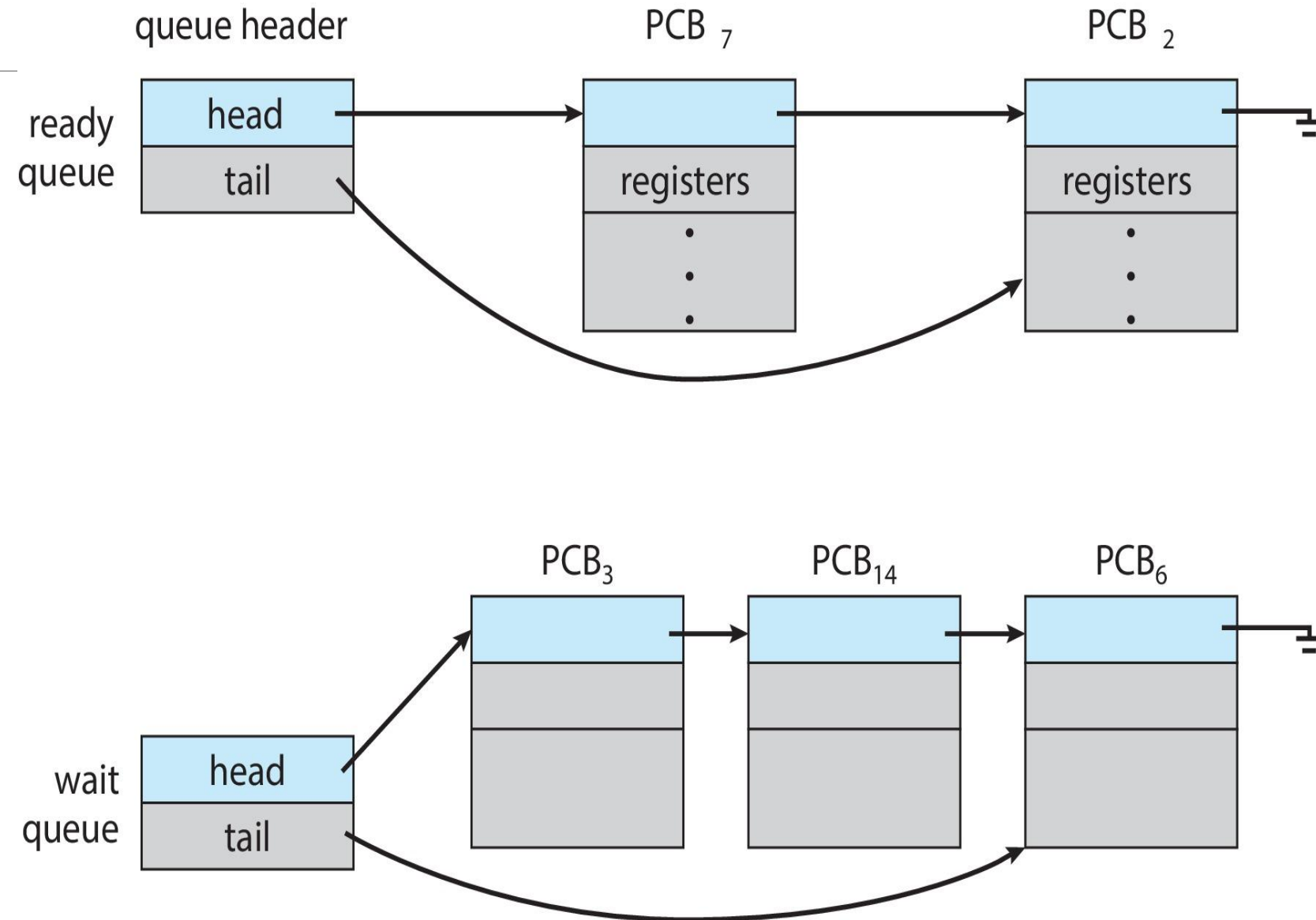
Goal -- Maximize CPU use, quickly switch processes onto CPU core

Maintains **scheduling queues** of processes

- **Ready queue** – set of all processes residing in main memory, and **are ready and waiting** to execute are kept on a list called the ready queue.
- **Wait queues** – The list of processes waiting for a **particular I/O device** is called as **device queue** or **wait queue**.
- Processes migrate among the various queues

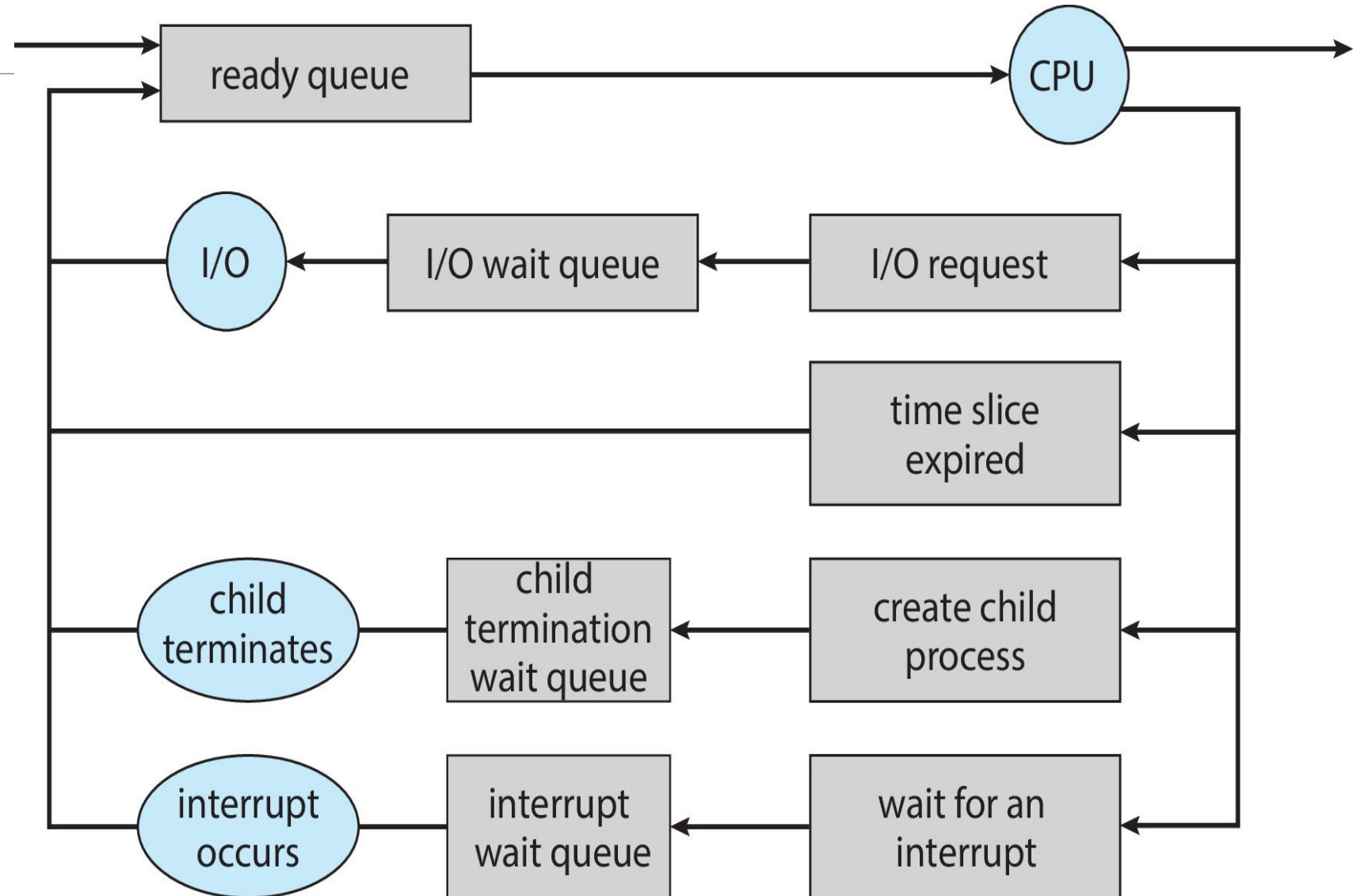
Ready and Wait or device Queues

- This queue is generally stored as Linked list.
- A **Queue header** contains pointers to the first & final PCBs in the list.
- Each PCB includes a **pointer** field that points to the **next PCB in the ready queue**.



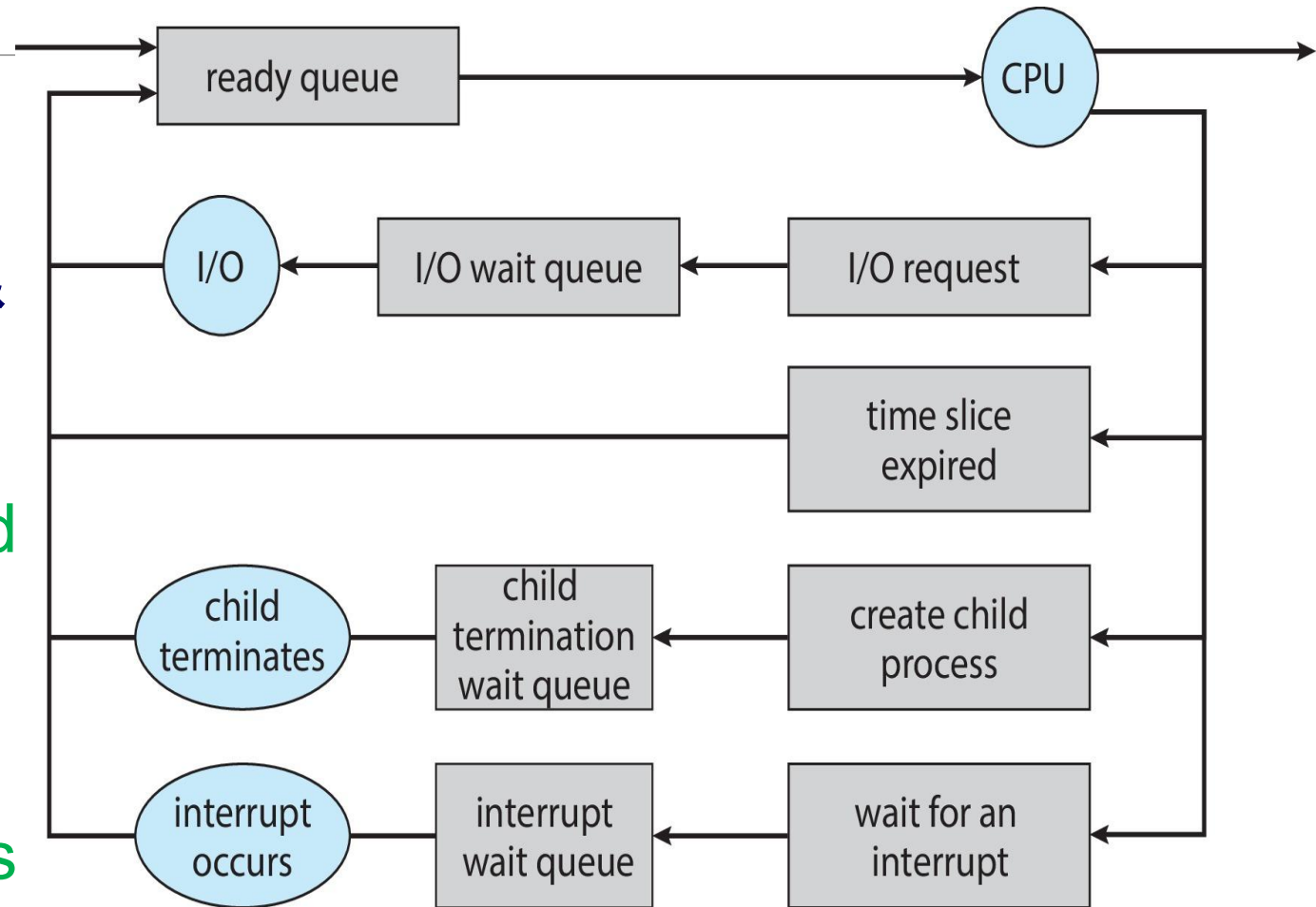
Queueing diagram- Representation of Process Scheduling

- Two types of queues are present:
 - Ready queue**
 - a set of device queues.**
- Rectangular box** represents a **queue**.
- Circles** represent the **resources** that serve the queues,
- The **arrows** indicate the **flow of processes** in the system.



Queueing diagram- Representation of Process Scheduling

- A new process is initially put in the ready queue.
- It waits there until it is selected for execution, or **dispatched**.
- Once the process is allocated the **CPU** & **is executing**, one of several events could occur: the process could
 - Issue an I/O request & then be placed in an I/O queue.
 - Create a new child process & wait for the child's termination.
 - Be removed forcibly from the CPU, as a result of an interrupt, & be put back in the ready queue.



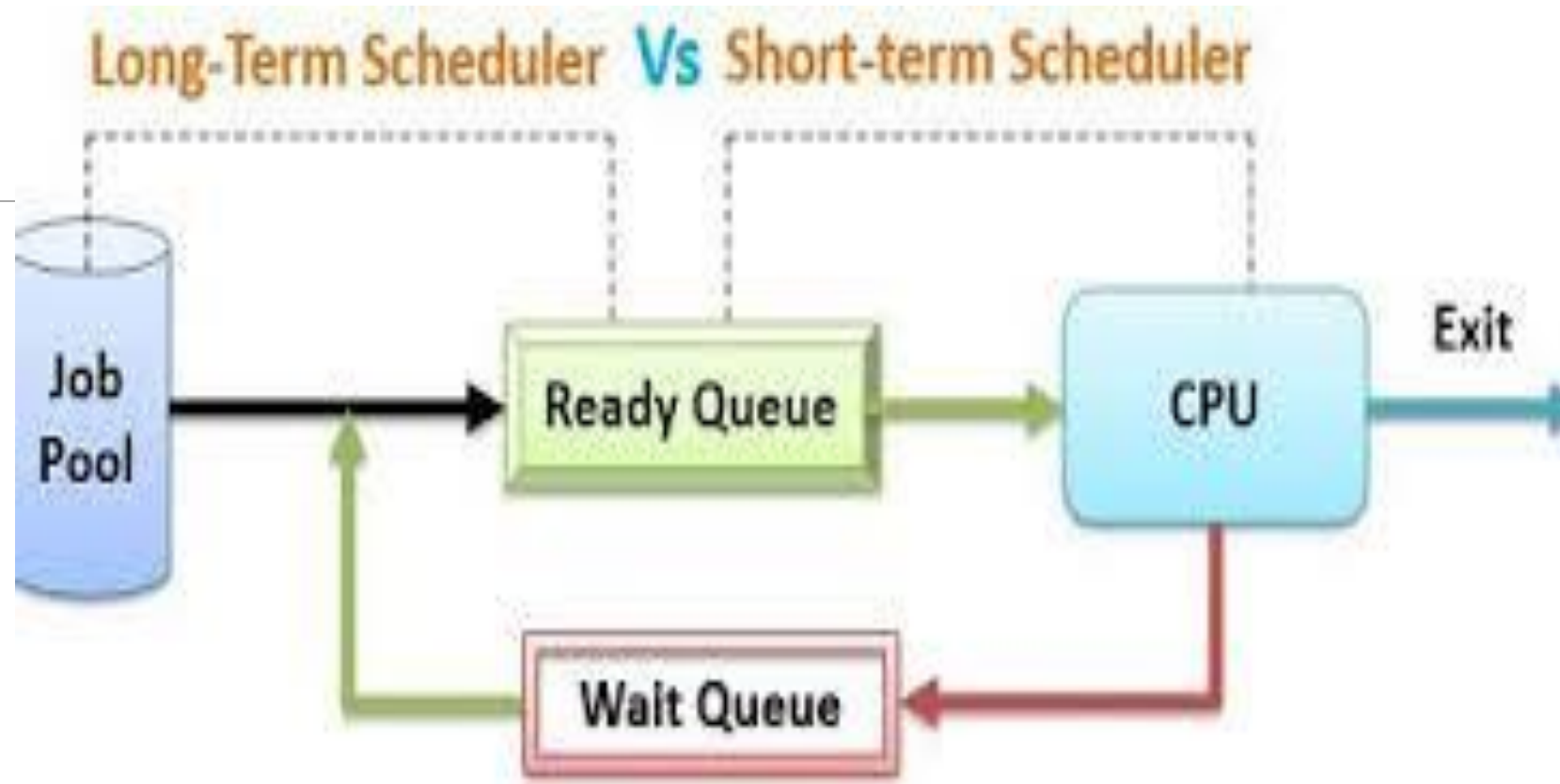
Schedulers

- A **process migrates** between various queues during its life time.
- OS selects processes for scheduling purpose from this queues in some fashion.
- This **selection process** is carried out by **scheduler**.
- Schedulers are special **system software** which handle process **scheduling** in various ways.
- Main task of scheduler is to select the **jobs to be submitted into the system** and to **decide which process to run**.

Types of Schedulers

1. Short-Term Scheduler
2. Long-Term Scheduler
3. Medium-Term Scheduler

Types of schedulers



- The **Short-term scheduler**, or **CPU scheduler**, select the processes that are ready to execute & allocates the CPU.
- The **Long-term scheduler**, or **Job scheduler**, selects processes from this job pool & loads them into memory for execution.

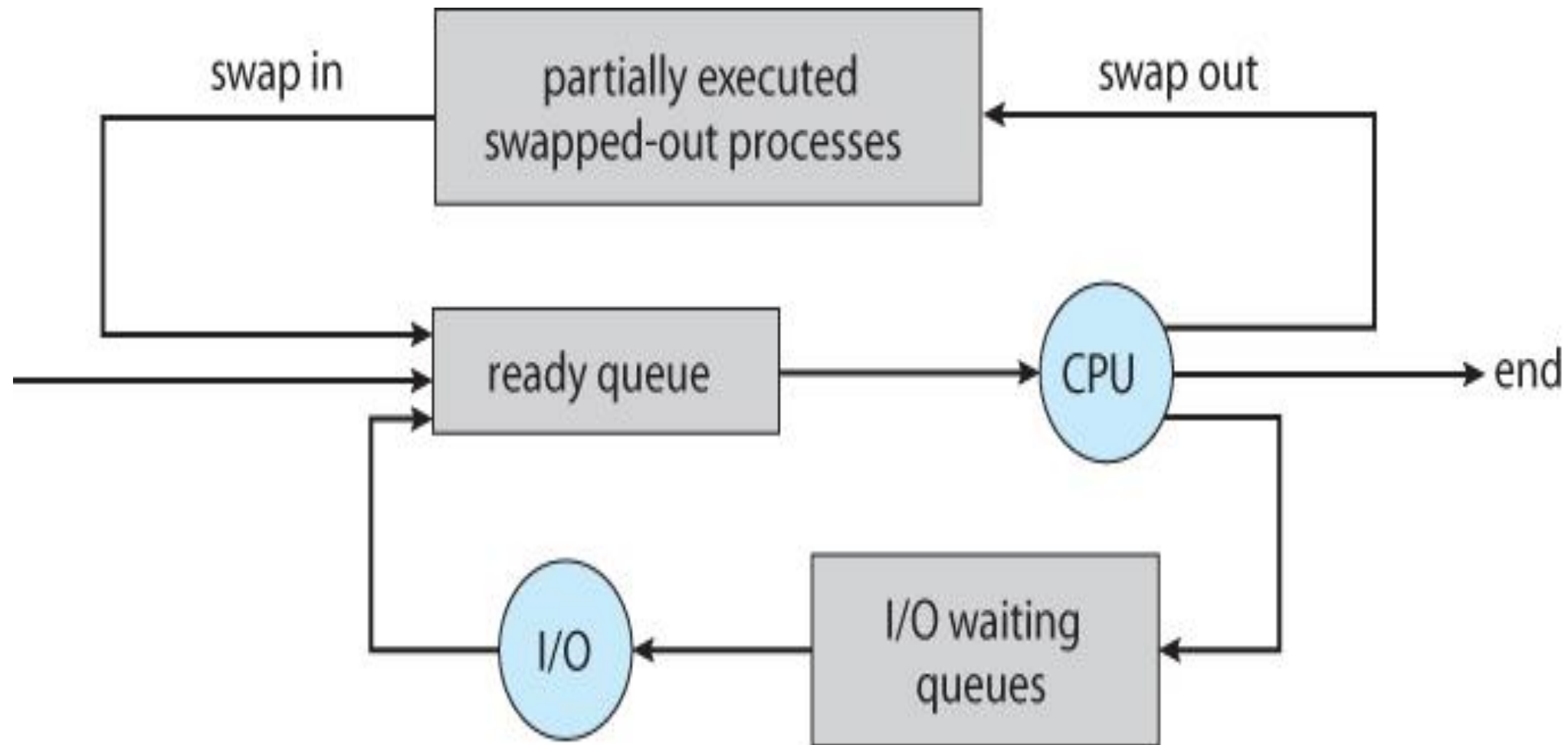
1. Short Term Scheduler

- It is also called as **CPU scheduler**.
- Its main objective is to increase system performance in accordance with the chosen set of criteria.
- It is the change of **ready state to running state** of the process.
- CPU scheduler selects a process among the processes that are ready to execute and allocates CPU to one of them.
- Short-term schedulers, also known as **dispatchers**, make the decision of which process to execute next.
- Short-term schedulers are faster than long-term schedulers.

2. Long Term schedulers

- It is also called a **job scheduler**.
- It determines which programs are admitted to the system for processing. It selects processes from the queue and loads them into memory for execution.
- The primary **objective of the job scheduler** is to provide a balanced mix of jobs, such as I/O bound and CPU bound.
- It also controls the degree of multiprogramming.
- On some systems, the long-term scheduler may not be available or minimal. Time-sharing operating systems have no long term scheduler.
- When **a process changes the state from new to ready**, then there is use of long-term scheduler.

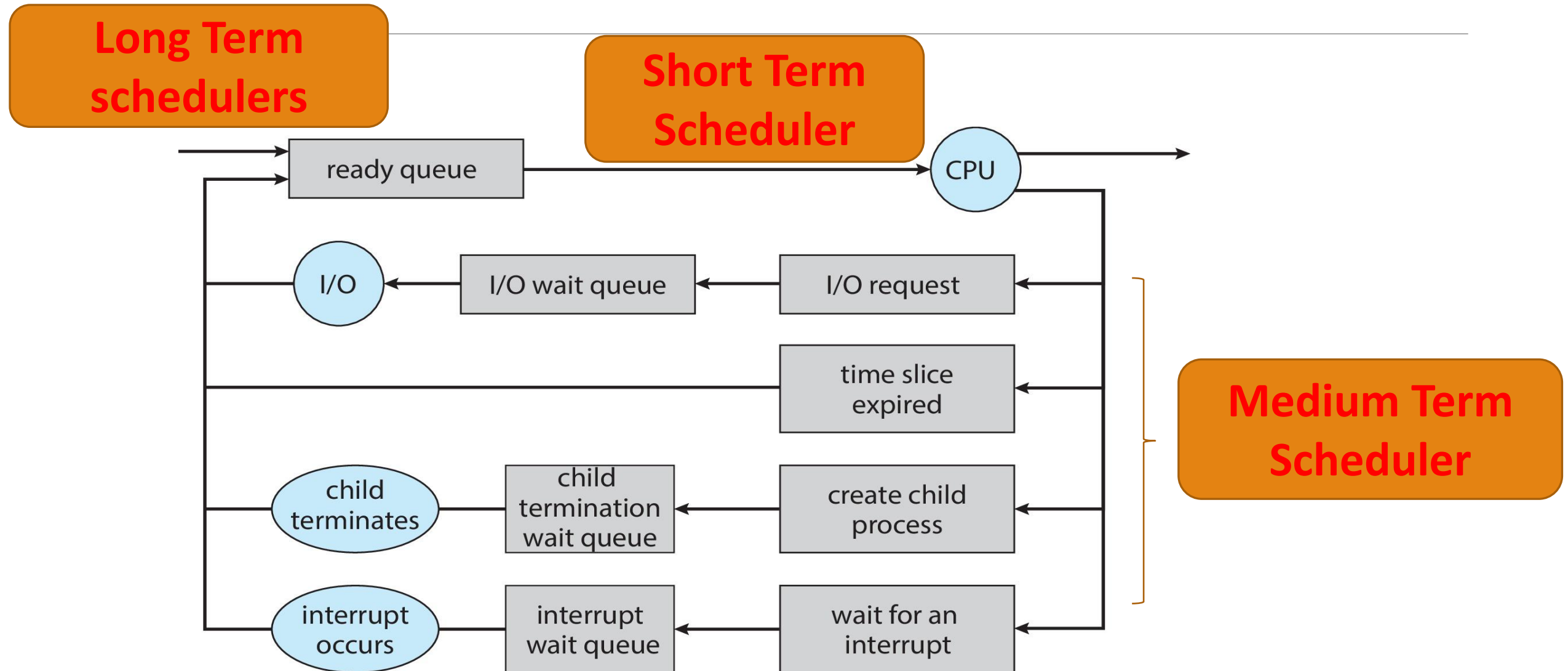
Medium Term schedulers



3. Medium Term Scheduler

- Medium-term scheduling is a part of swapping.
- It removes the processes from the memory. It reduces the degree of multiprogramming.
- The medium-term scheduler is in-charge of handling the swapped out-processes.
- A running process may become suspended if it makes an I/O request. A suspended processes cannot make any progress towards completion. In this condition, to remove the process from memory and make space for other processes, the suspended process is moved to the secondary storage. This process is called swapping, and the process is said to be swapped out or rolled out.
- Swapping may be necessary to improve the process mix.

Types of schedulers



Difference between Schedulers

Sr. No.	Long Term Scheduler	Medium Term Scheduler	Short Term Scheduler
1.	It is also called as Job Scheduler	It is a process of swapping	It is also called as CPU Scheduler
2.	It's Speed is less than Short term scheduler	Its speed is in between short term & long terms scheduler.	It is faster than Long term scheduler
3.	It control degree of multiprogramming.	It reduces degree of multiprogramming.	It provides lesser control over degree of multiprogramming.
4.	When a process changes the state from new to ready , then there is use of long-term scheduler	When a process changes the state from running to waiting , then there is use of Medium-term scheduler	When a process changes the state from ready to running , then there is use of short-term scheduler
5	It is absent in time sharing OS	It is part of time sharing OS	It is also minimal in time sharing OS
6.	It select process from job queue and load them into main memory for execution.	It performs swapping operation on partially executed process	It selects process from ready queue and allocate CPU.

Interprocess Communication

- Processes **executing concurrently** in the OS may be either **independent processes** or **cooperating processes**.
 - **Independent Processes:-** They cannot affect or be affected by the other processes executing in the system.
 - **Cooperating process** can affect or be affected by other processes, including sharing data.
- Reasons for cooperating processes:
 - **Information sharing**
 - **Computation speedup** :-divide task to sub task & all sub task may execute concurrently. In that way will speed up our system.
 - **Modularity** :- divide the system into different modules later will put together to achieved a single goal.
 - **Convenience:-** Providing an environment that allows process cooperate to each other then that become very Convenience for the user.
- Cooperating processes require an **inter process communication (IPC)** mechanism that will allow them to exchange data & information.
- There are Two fundamental models of IPC
 - **Shared memory**
 - **Message passing**

Interprocess Communication

1. Shared memory:-

- It can be referred to as a type of memory that can be used or accessed by multiple processes simultaneously.
- It is primarily used so that the processes can communicate with each other.
- Therefore the shared memory is used by almost all POSIX and Windows operating systems as well.

2. Message passing

- It is a type of mechanism that allows processes to synchronize and communicate with each other. However, by using the message passing, the processes can communicate with each other without restoring the shared variables.

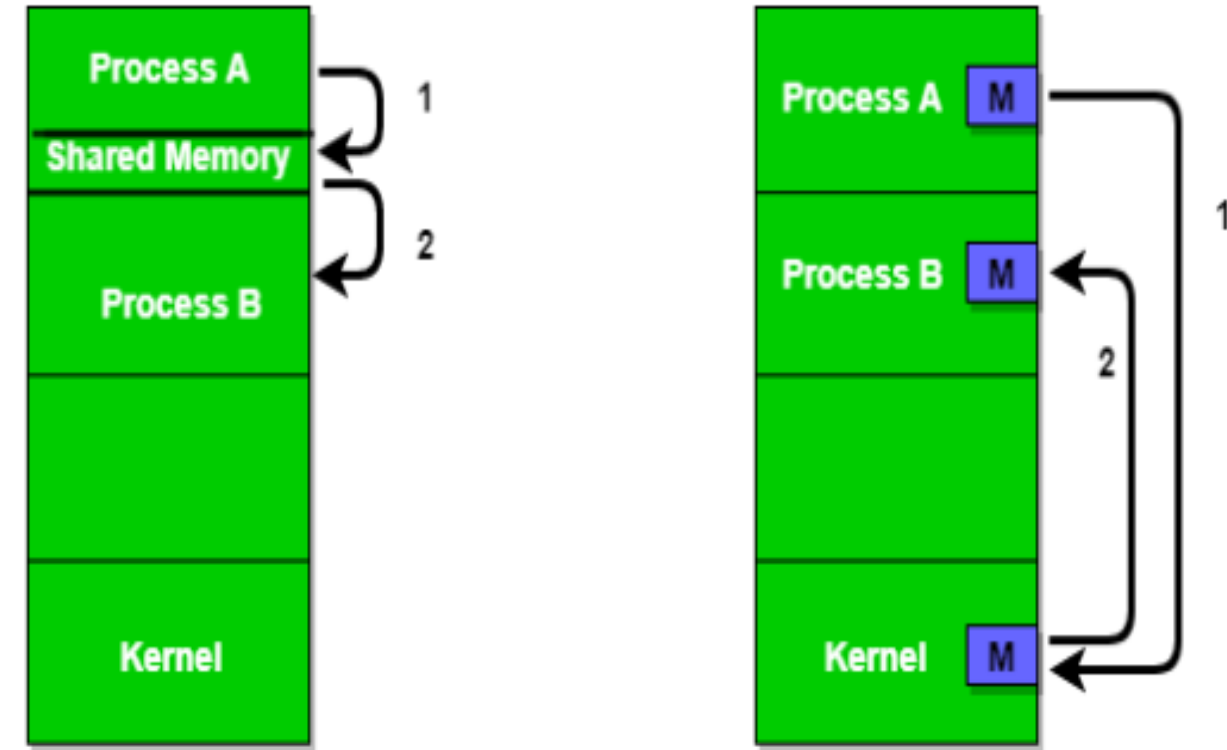
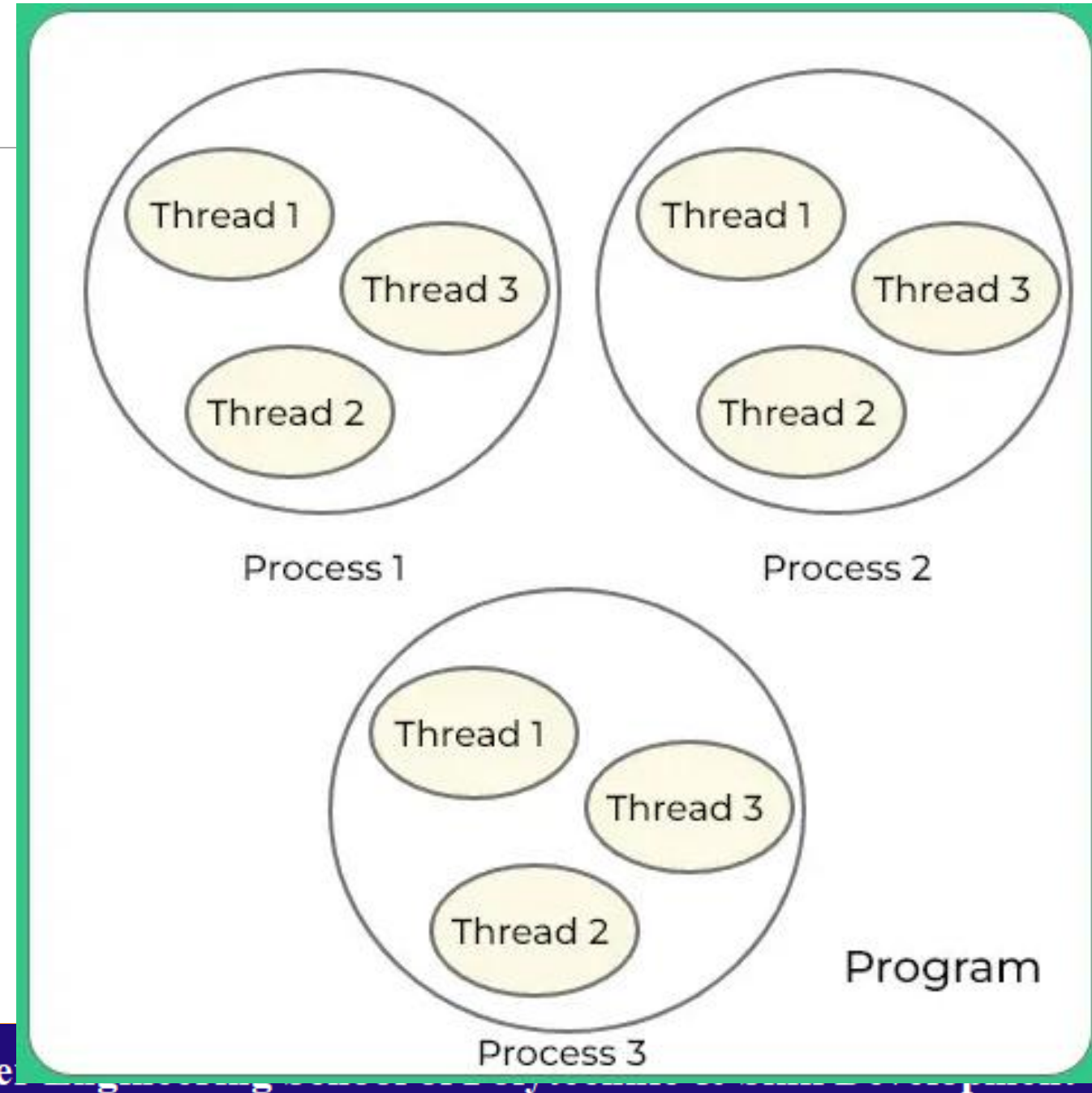
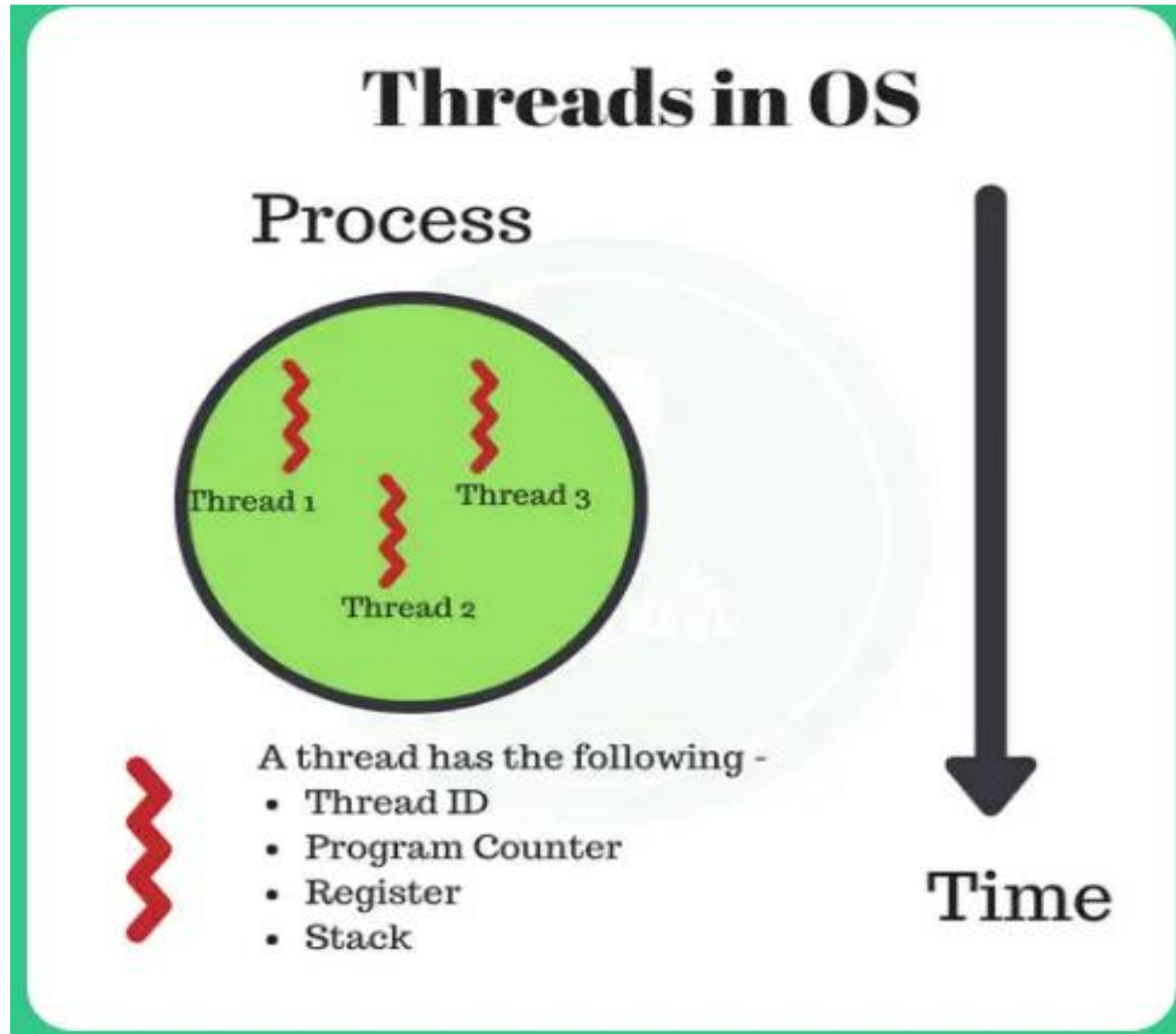


Figure 1 - Shared Memory and Message Passing

Thread Concept:

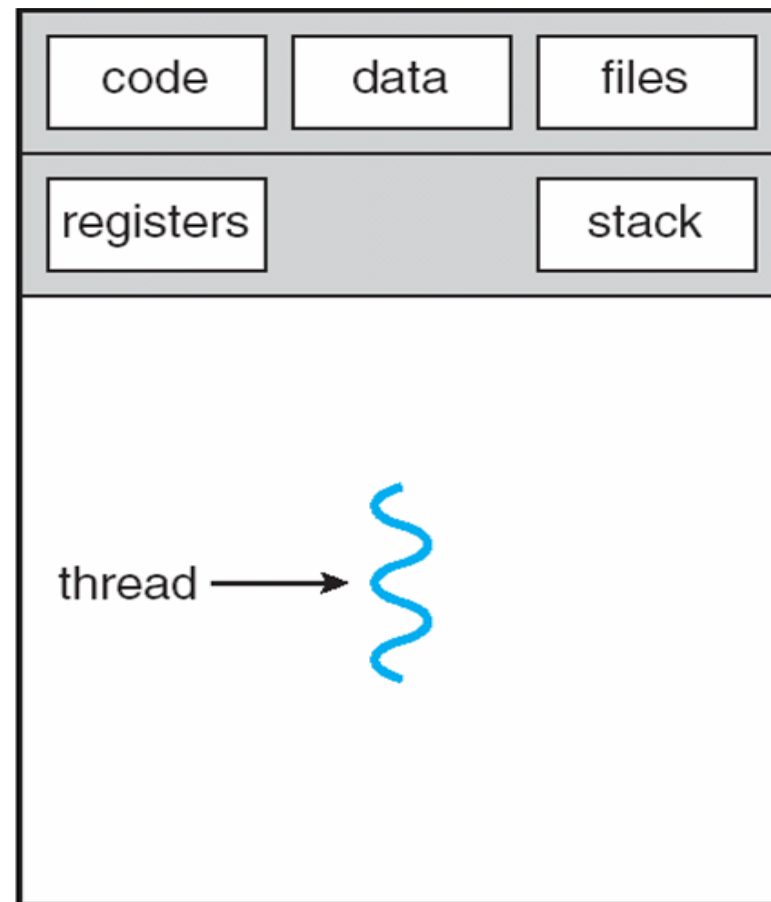
- A **thread** refers to a **single sequential flow of activities** being executed in a process; it is also known as the **thread of execution** or the **thread of control**.
- Thread is the **smallest executable unit** of a **process**. For example, when you run a notepad program, operating system creates a process and starts the execution of main thread of that process.
- A process can have **multiple threads**.
- Each thread will have their own task and own path of execution in a process.
- For example, in a notepad program, one thread will be taking user inputs and another thread will be printing a document.

Thread Concept:

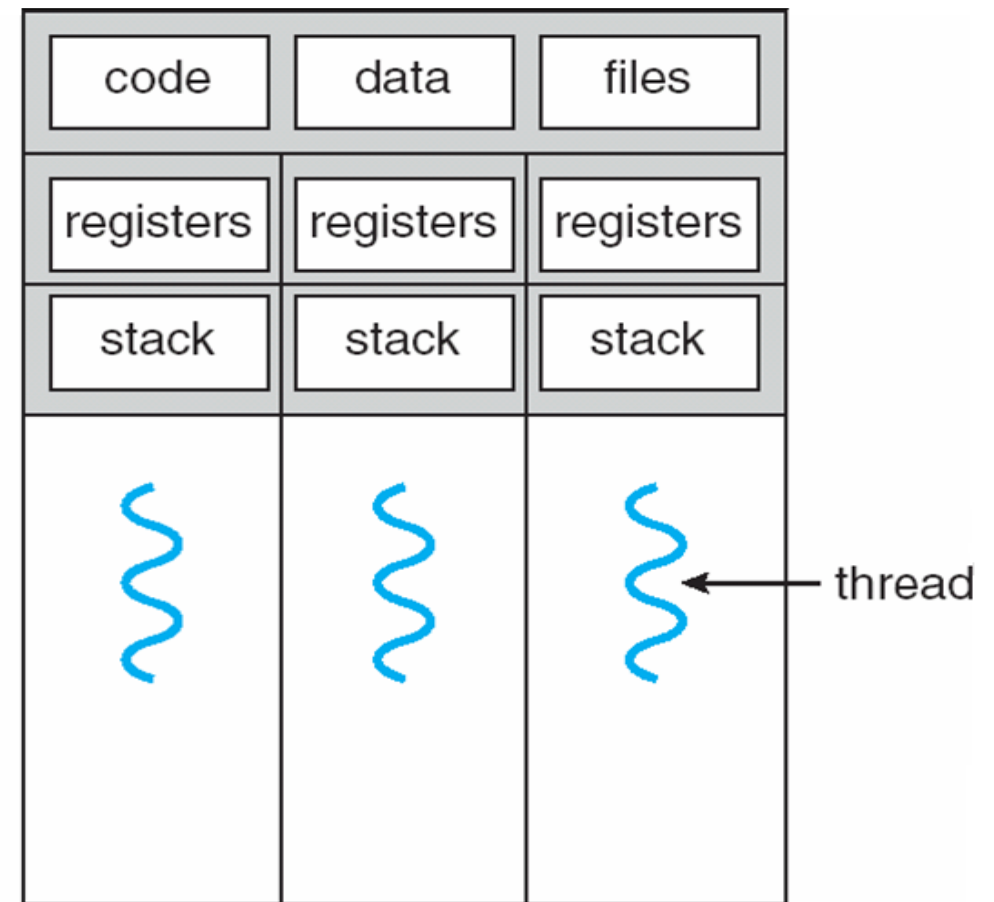


Multithreading Models:

- There are two different types of processes
 - Single-threaded processes
 - Multi-threaded processes



single-threaded process



multithreaded process

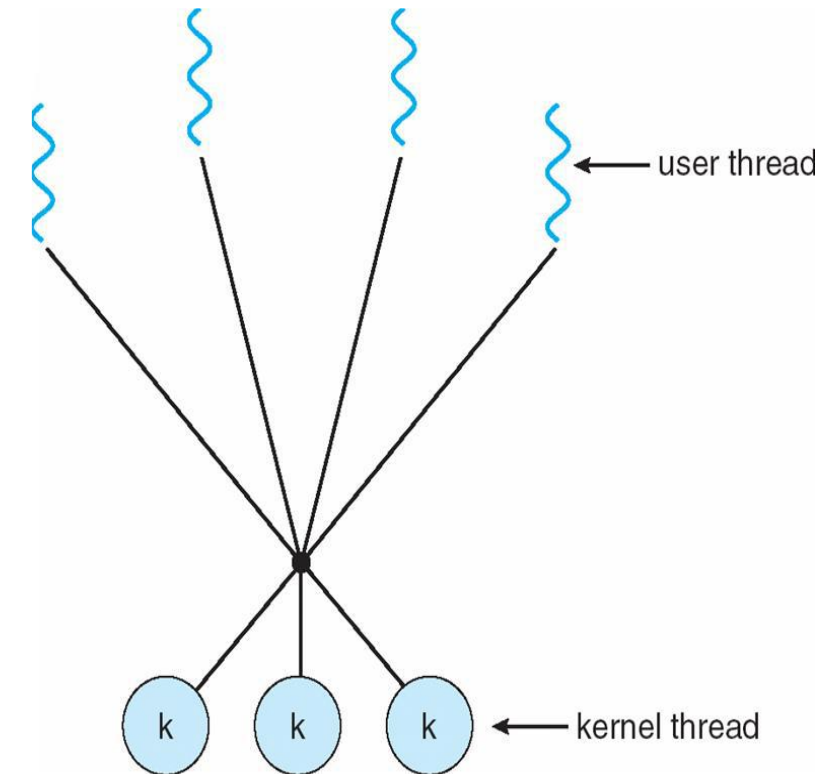
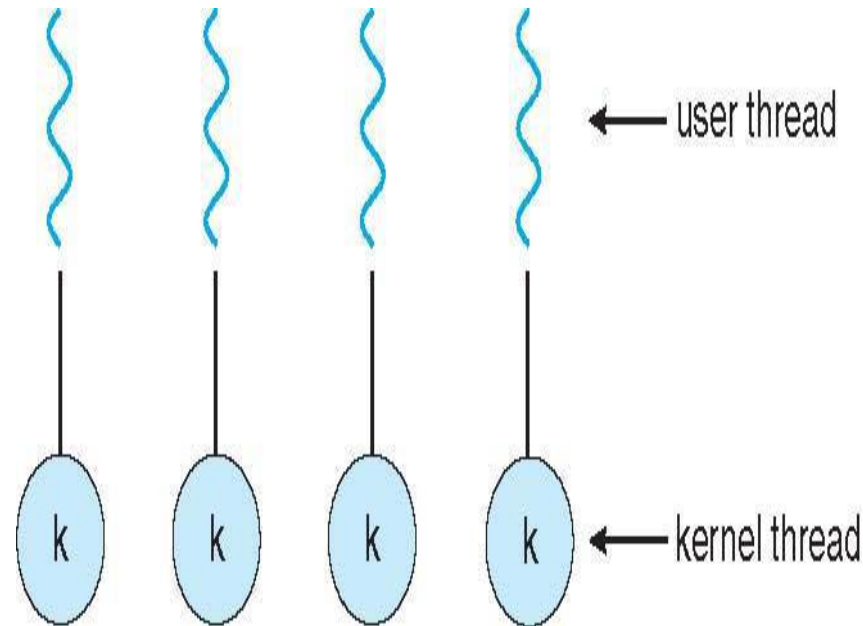
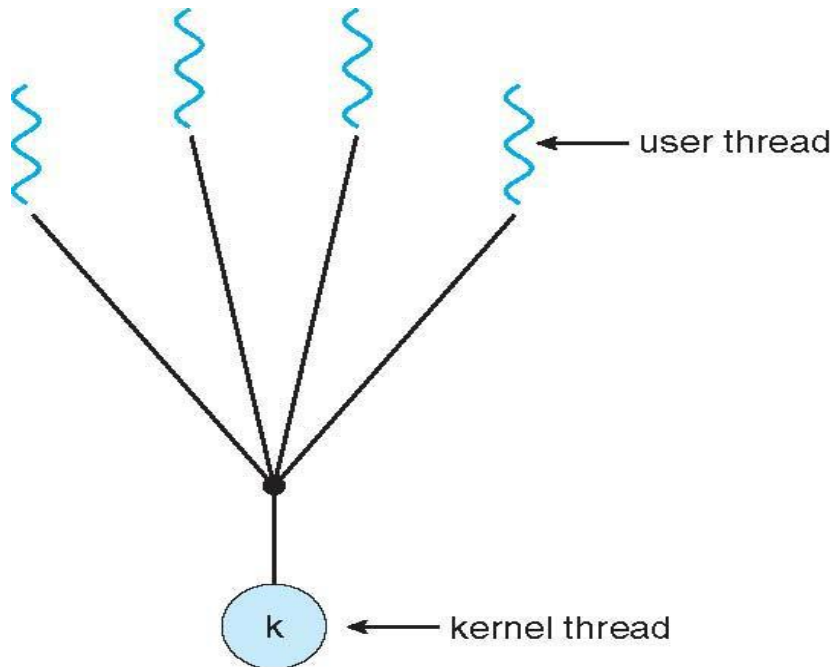
Multithreading Models:

Multi-threaded application running on a traditional single-core chip

- A thread is also known as a lightweight process.
- The idea is to achieve parallelism by dividing a process into multiple threads.
- 1. Example, in a browser, multiple tabs can be different threads.
- 2. Example:- MS Word uses multiple threads: one thread to format the text, another thread to process inputs, etc.
- **Multithreading** is a technique used in operating systems to **improve the performance and responsiveness of computer systems**.
- Multithreading allows **multiple threads** (i.e., lightweight processes) to share the **same resources of a single process**, such as the CPU, memory, and I/O devices.

Multithreading Models

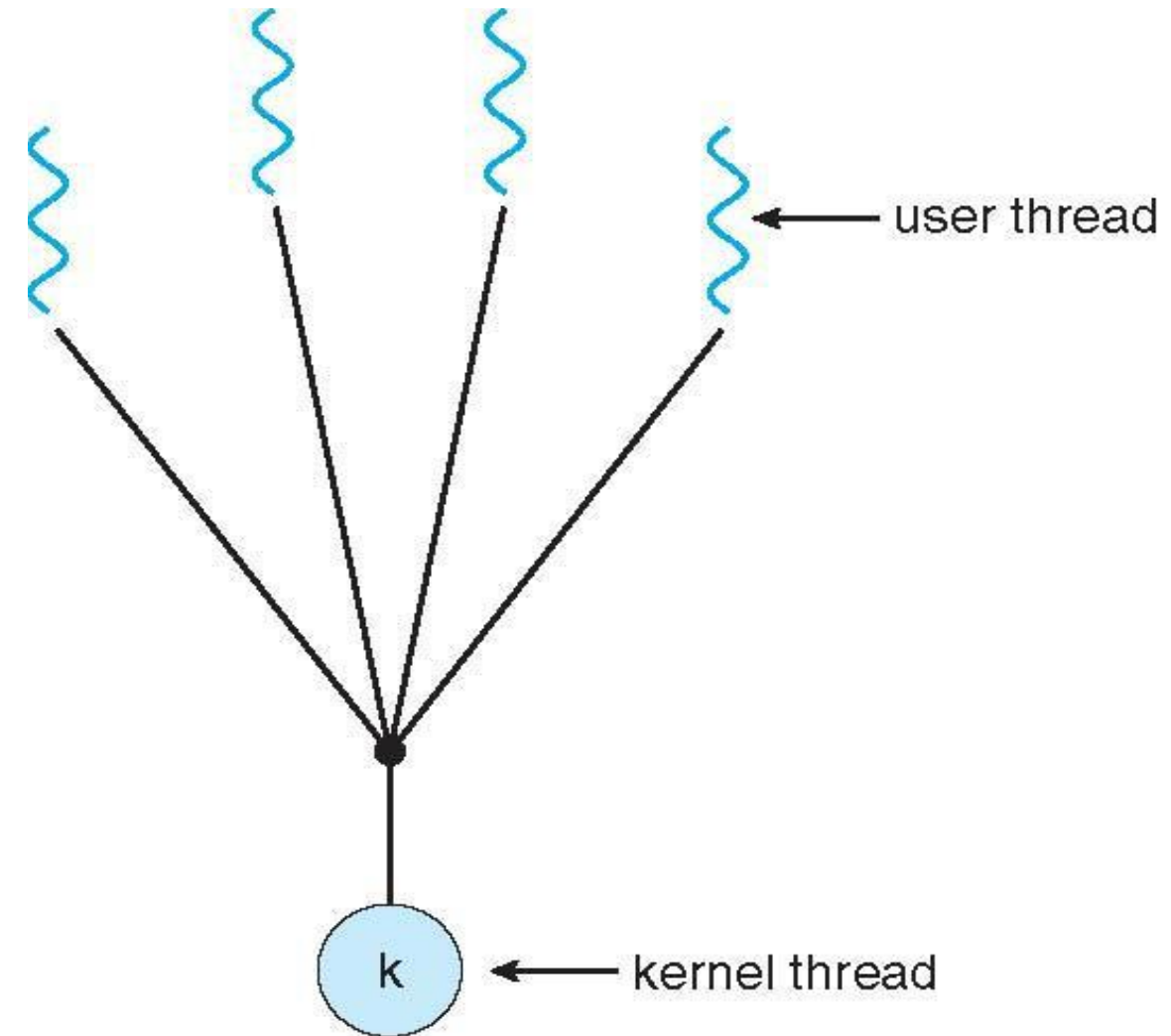
1. Many to one model
2. One to one model
3. Many to many model



Multithreading Models

1. Many to one model

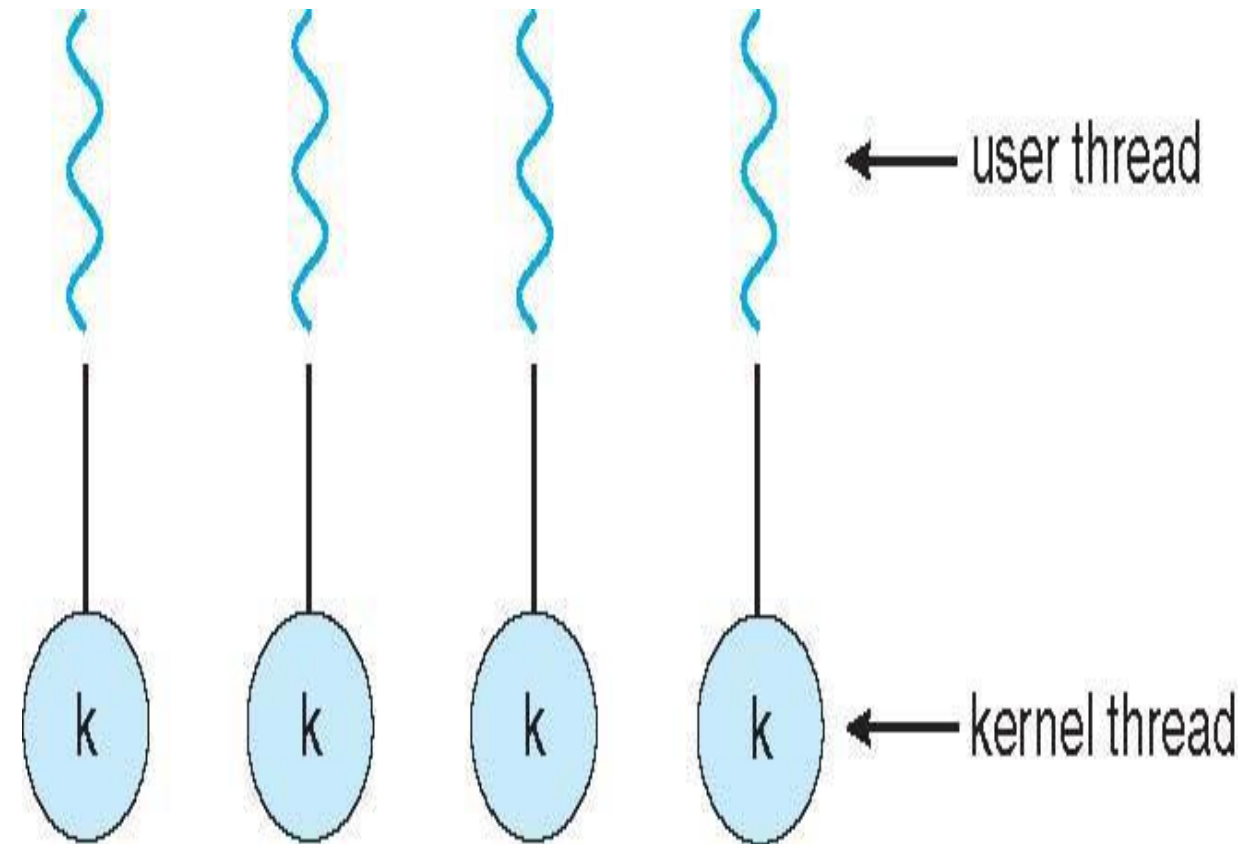
- In the many-to-one model, **many user-level threads are all mapped onto a single kernel thread.**
- Thread management is handled by **the thread library in user space, which is very efficient.**



Multithreading Models

2. One to One model

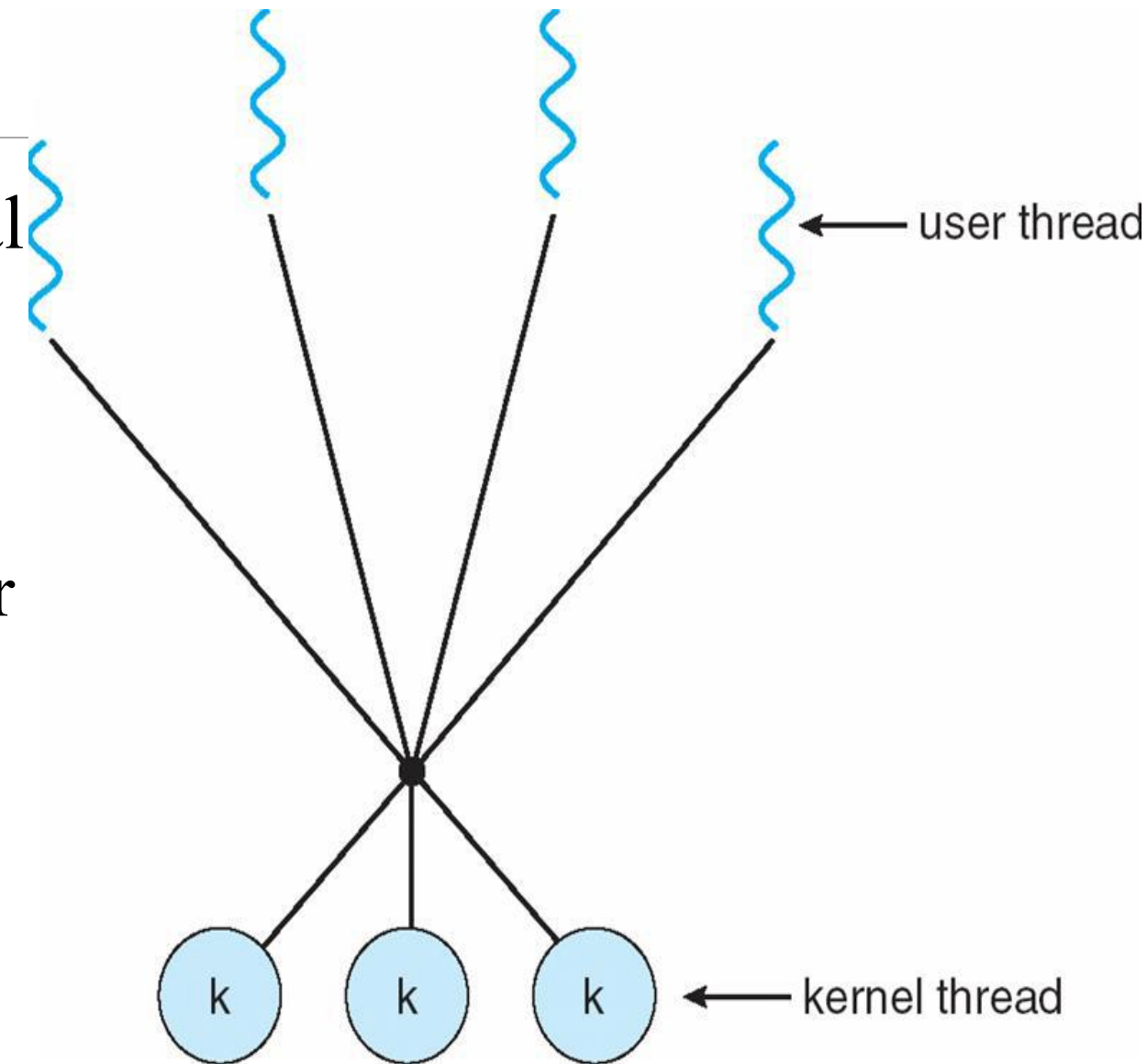
- The one-to-one model creates a separate **kernel thread to handle each user thread**.
- One-to-one model **overcomes the problems listed above involving blocking system calls and the splitting of processes across multiple CPUs**.



Multithreading Models

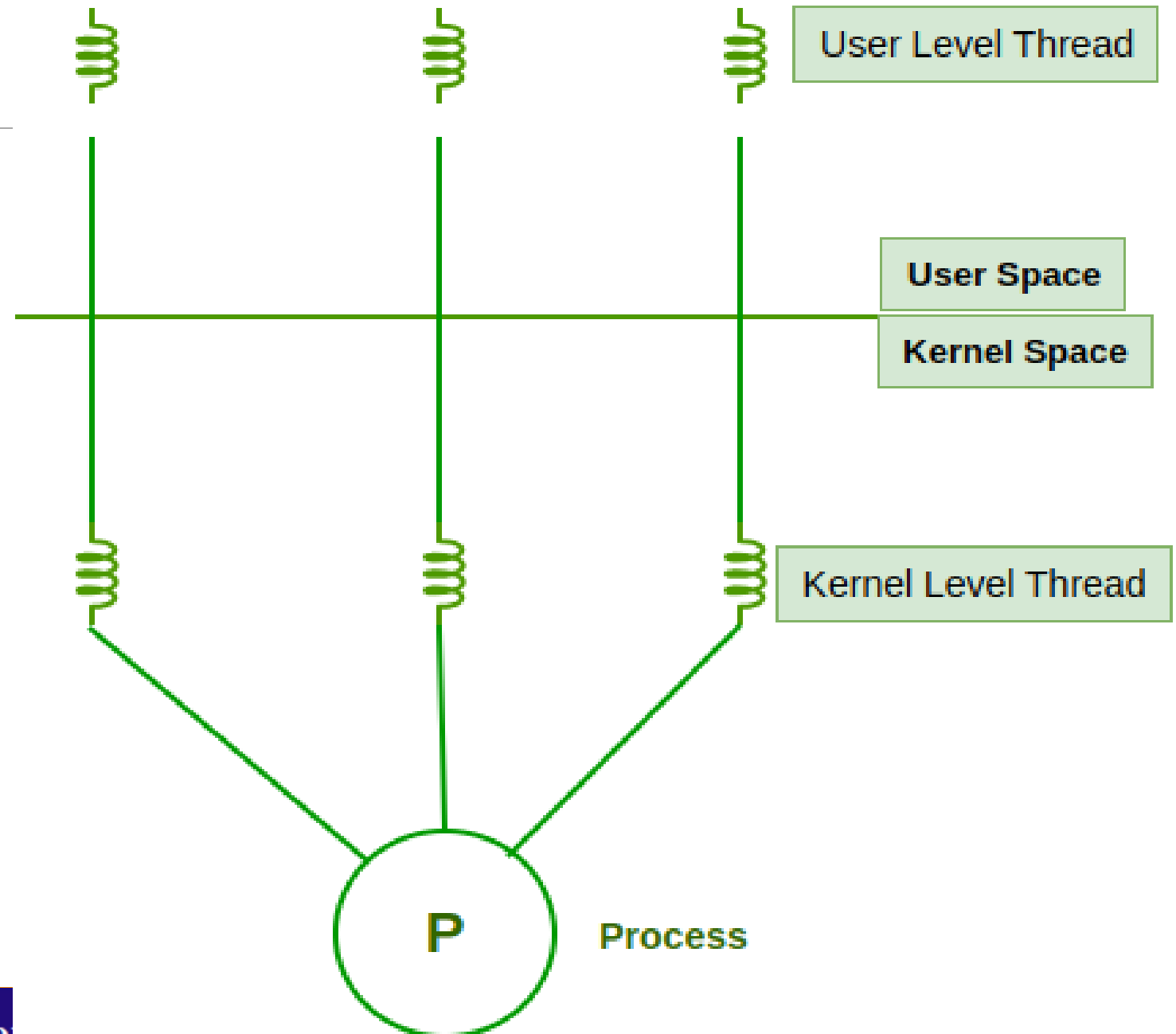
1. Many to Many model

- The many-to-many model multiplexes any number of user threads onto an equal or smaller number of kernel threads, combining the best features of the one-to-one and many-to-one models.
- Users have no restrictions on the number of threads created.
- Blocking kernel system calls do not block the entire process.
- Processes can be split across multiple processors.



Types of Thread in Operating System

1. User Level Thread
2. Kernel Level Thread



1. User Level Thread

Features of User Level Thread –

- Implemented by user and managed by run-time system (user-level library).
- OS is not aware of existence of threads.
- OS manages the user level threads as if they are single threaded process.
- Creation of thread switching between thread and synchronizing thread are all done via procedural call.

Advantage of User Level Thread-

- Are extremely fast and efficient, because the switching between thread takes almost same time as a procedural call.
- OS doesn't need to interfere ,everything can be without it.
- They do not modify OS.
- Threads are cheap and fast at user level.

Disadvantage of User Level Thread-

- Since OS isn't aware of user level thread , scheduler does not schedule them properly.
- If 1 user level thread performs blocking operation, entire process will be blocked.

2. Kernel Level Thread

Features of Kernel Level Thread –

- OS managed threads.
- OS knows about and manages the thread.
- Kernel has a thread table which keeps tracks of all threads in the system.
- System calls create and manage threads.

Advantages of Kernel Level Thread ->

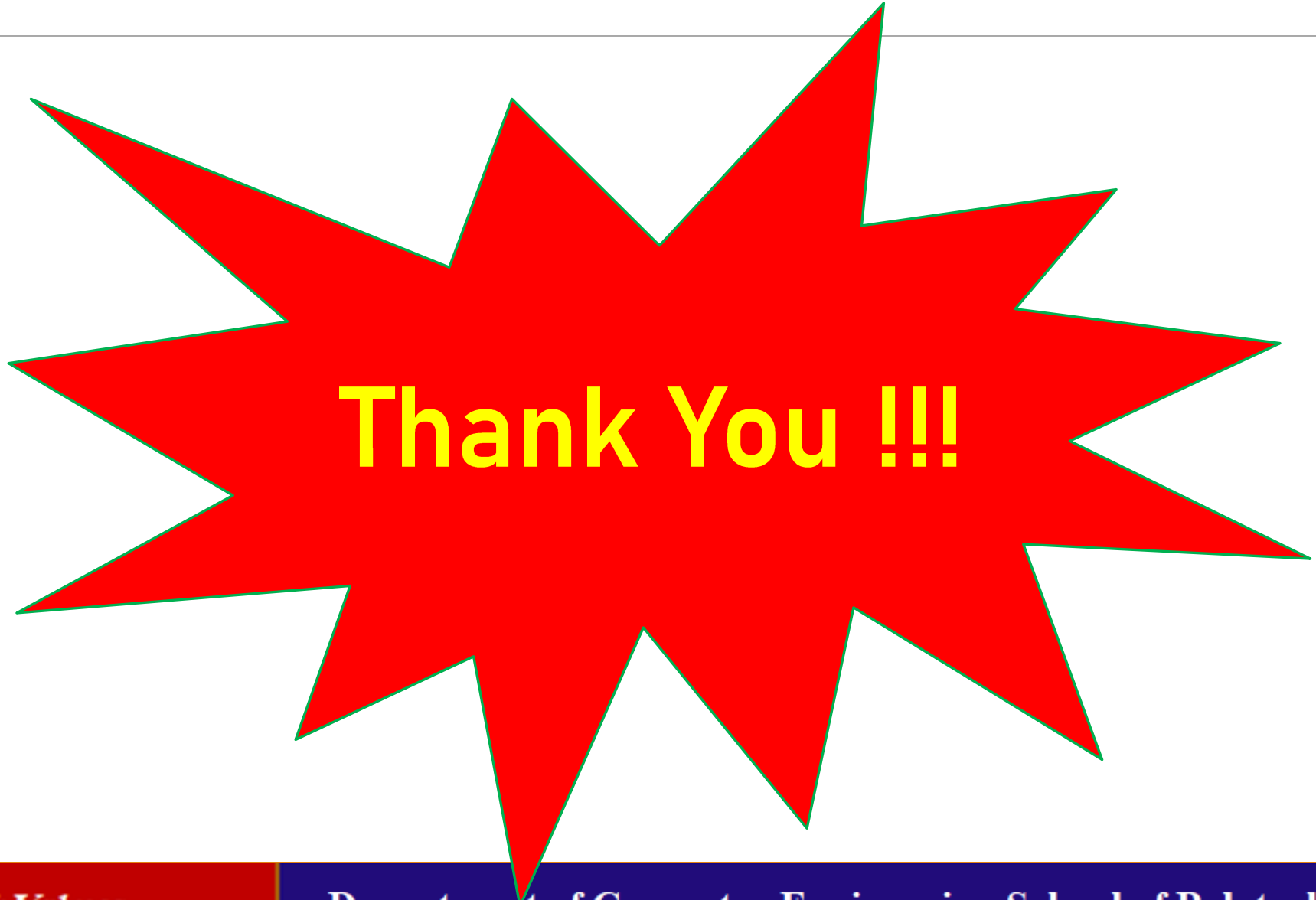
- As kernel has full knowledge of threads, so scheduler handles the processes better.
- For application that frequently block, Kernel level threads are good.

Disadvantages of Kernel Level Thread ->

- Slow and inefficient than user level.
- Hardware support is needed.
- Context switching time is more. Eg – Window Solaris
- If 1 kernel thread performs blocking operation, then another thread can continue execution.

User level Threads vs Kernel Level Threads

User level Threads	Kernel Level Threads
User thread are implemented by Users	Kernal threads are implemented by OS
OS doesn't recognise user level threads	Kernel threads are recognized by OS
Implementation is easy	Implementation is complicated
Context switch time is less	Context switch time is more
Context switch – no hardware support	hardware support is needed
If one user level thread perform blocking operation then entire process will be blocked	If one kernel level thread perform blocking operation then another thread can continue execution.



Thank You !!!